

# Práctica 1: Refactoring

martes, 31 de marzo de 2026 08:20

## Ejercicio 1: Algo huele mal

### 1.1 Protocolo de Cliente

Malos olores detectados: Nombres pocos descriptivos en los métodos

Los métodos son los siguientes:

- `public double lmtCrdt(){`
- `protected double mtFcE(LocalDate f1, LocalDate f2){`
- `private double mtCbE(LocalDate f1, LocalDate f2){`

Solución:

#### Aplicar Rename Method

Renombrar los métodos para hacer sus nombres más claros y evitar el uso de excesivo de comentarios

- `public double limiteCredito()`
- `protected double montoFacturado(LocalDate f1, LocalDate f2)`
- `private double montoCobrado(LocalDate f1, LocalDate f2)`

## Ejercicio 1.2:

Code smells detectados:

- Envidia de atributos
- Asigna mal responsabilidades
- Rompe encapsulamiento

Se realizó un **Refactoring Move Method**, me parece apropiado ya que mueve el método hacia la clase donde se encuentran los servicios que usa y decide borrarlo de la clase **Persona**, ya que no es necesario mantenerlo para hacer una delegación en este caso.

## Ejercicio 1.3:

Code smells detectados:

- Long Method
- Excesivo uso de variables temporales

- Imperative Loops

**Solución:** Aplicar los siguientes Refactorings

Especificar donde muevo las cosas y todo lo que hago en cada refactoring. Osea que loop voy a reemplazar, etc.

- **Replace Loop With Pipeline**
- **Extract Method**
- **Replace Temp with Query**

**Ejercicio 2:**

### **Ejercicio 3:**

**Aplicar Extract Superclass**

1. Creo una superclase abstracta vacía y hago que las clases originales hereden de esta superclase.
2. Aplico **Pull Up Field** con idx
3. Aplico **Extract Method** sobre el código repetido en next()
4. Aplico **Pull Up Method** sobre el método nuevo que extraje y me quedó duplicado en las dos subclases.

### **Ejercicio 4:**

Aplicar **Rename Method** a **procesar()**:

- Declaro un nuevo método con el nuevo nombre, copio el cuerpo del antiguo método en el nuevo, hago las modificaciones necesarias si las hay.
- Cambio el cuerpo del método antiguo para que llame al nuevo.
- Compilo y pruebo.
- Busco todas las referencias al antiguo método y las cambio para que se refieran al nuevo. Compilo y pruebo después de cada cambio.
- Elimino el método antiguo
  - Si el método antiguo forma parte de la interfaz y no puedes eliminarlo, déjalo en su lugar y márcalo como **deprecated**.

Aplicar **Rename Method** a **calcular()**:

- Declaro un nuevo método con el nuevo nombre, copio el cuerpo del antiguo método en el nuevo, hago las modificaciones necesarias si las hay.
- Cambio el cuerpo del método antiguo para que llame al nuevo.
- Compilo y pruebo.
- Busco todas las referencias al antiguo método y las cambio para que se refieran al nuevo. Compilo y pruebo después de cada cambio.
- Elimino el método antiguo
  - Si el método antiguo forma parte de la interfaz y no puedes eliminarlo, déjalo en su lugar y márcalo como **deprecated**.

Aplicar **Rename Method** a **calcular()** en clase **Perfil**:

- Declaro un nuevo método con el nuevo nombre, copio el cuerpo del antiguo método en el nuevo, hago las modificaciones necesarias si las hay.
- Cambio el cuerpo del método antiguo para que llame al nuevo.
- Compilo y pruebo.
- Busco todas las referencias al antiguo método y las cambio para que se refieran al nuevo. Compilo y pruebo después de cada cambio.
- Elimino el método antiguo
  - Si el método antiguo forma parte de la interfaz y no puedes eliminarlo, déjalo en su lugar y márcalo como **deprecated**.

**Problema:** Me queda el mismo nombre que el método en la clase Publicación, debo volver a aplicar Rename Method sobre el método en clase perfil?

Aplicar **Rename Parameter** a "p":

- Renombro el parámetro
- Renombro todas sus referencias dentro del método.

## **Ejercicio 5:**

### **5.1)**

- Declarar getters y setters para la variable.
- Buscar/Encontrar todos los usos de la variable por fuera de la clase y reemplazarlos por el getter o setter según corresponda.
- Compilar y probar
- Una vez que todos los usos estén cambiados, declarar la variable como privada.
- Compilar y probar.

**5.1.a)** Si, es correcto mientras no modifique el comportamiento del Test.

**5.1.b)** En caso de que el test falle, puede indicar que se ha cambiado el comportamiento en el código. O que no se aplicó bien el refactoring y han quedado referencias a la variable sueltas sin reemplazar por los getters y setters.

**5.2)** No encontré los pasos exactos en el libro de este refactoring, pero hay que renombrar el campo y todas sus invocaciones desde cualquier otra clase y dentro de ella misma también. Sí es necesario modificar los test, tan solo los llamados a dicha variable que fue modificada. Pero no habría inconveniente, ya que no estamos modificando el comportamiento.

### **Rename Parameter?**

**5)3)**

Sí es posible hacerlo con el código anterior debido a que cumple con las precondiciones impuestas en dicho Refactoring, pero antes debemos aplicar **Pull Up Field** sobre la variable timePeriod y luego Pull Up Constructor Body.

**5)4)**

Lo mencionado en el punto anterior.

**5)5)**

**5)6)**

### **Malos olores:**

- Código duplicado en los nombres de los métodos
- Feature Envy a Hotel y Company

### **Solución:**

### **Move Method??, Rename Method, Extract Method o Pull Up Method?**

Aplico **Move Method** a **price()** en carRental para delegarlo a Company, dejo el método anterior.  
Aplico **Rename Method** a **price()** en Company a **calcularPrecio()**

Aplico **Move Method** a cierta parte de **price()** en HotelStay para delegarlo a Hotel, dejo el método anterior como delegación.

Aplico **Rename Method** a **price()** en Company a calculoPrecio()

### **Ejercicio 6.1:**

#### **Bad smells:**

- Código duplicado (en v.i)
- Clases con características similares
- Rompe encapsulamiento (v.i públicas).
- Magic numbers

#### **Solución:**

Aplicar **Encapsulate Field**

Aplicar **Extract Superclass**

Aplicar **Form Template Method**

Aplicar **Replace Magic Numbers with Symbolic Constant**

### **Ejercicio 6.2:**

#### **Bad smells:**

- Public Field. Rompe encapsulamiento (v.i públicas). Otra clase modifica su estado interno de la clase Jugador.
- Feature Envy

#### **Solución:**

 Aplicar **Encapsulte Field** (nombre, apellido y puntuación)  
Aplicar **Move Method** (incrementar y decrementar a clase Jugador)

### **Ejercicio 6.3:**

#### **Bad smells:**

- Long Method
- Imperative Loops

- Comments

### Solución:

Aplicar **Replace Loop with Pipeline**, con eso ya me alcanza?

### **Ejercicio 6.4:**

#### Bad smells:

- **Feature Envy** (en clase carrito, método total)
- **Message Chains/Cadena de mensajes:** Muchos objetos intermediarios entre el que origina la llamada hasta el que responde el mensaje, Carrito tiene items e Item tiene un Producto, Carrito conoce toda esta estructura y esta acoplado a ella. Un cambio en la estructura implica un cambio en la cadena de mensajes.

### Solución:

- **Move Method** (una parte del método total de carrito, la que le corresponde a item, delego).
- **Rename Method** (método **total()** en clase Carrito a **calcularCompra()**, para que no sea ambiguo).

### **Ejercicio 6.5:**

#### Bad smells:

- **Feature Envy** (en clase cliente sobre dirección)
- **Public Field** (en clase direccion)
- **Comments**

### Solución:

Aplicar **Encapsulate Field** (clase direccion)

Aplico **Move Method** a **getDirecciónFormateada()**, moverlo a clase dirección

Aplico **Rename Method** a **getDireccionFormateada()** en clase Dirección, para desambiguar.

Aplico **Extract Method** a **notificarPedido()** en clase Cliente

### **Ejercicio 6.6:**

#### Bad smells:

- **Switch Statements** (clase Usuario, método calcularCostoPelicula)
- **Comments**
- **Primitive Obsession** (para el tipoSubscripcion)
- **Magic Numbers**

### Solución:

Aplicar **Replace Conditional with Polymorphism** para tipoSubscripcion

Aplicar **Extract Method** en **calcularCargoExtraPorEstreno()** en clase Pelicula, para que sea más legible y poder sacar los comentarios.

Aplico **Rename Method** al método abstracto de la superclase y lo que calculan todos los tipos de subscripciones

Aplico **Replace Magic Number with Symbolic Constant**

### Ejercicio 7:

**7.1)** Sí, en el método **generar()**. En las líneas:

```
System.out.println("Producto: " + nombreProducto);
System.out.println("-----");
```

**7.2)** No es posible, debido a que los cuerpos de los métodos deben ser idénticos.

**7.3)** Se debe aplicar un **Extract Method** de los comportamientos comunes y luego aplicar un **Form Template Method**, ya que nos queda el código que no se repite en medio del código común de ambos comportamientos.

**7.4)**

Aplicar **Extract Method** sobre el comportamiento común.

Aplicar **Form Template Method** sobre generar(), para poder ponerlo en la superclase. Identificar los pasos que hace, **imprimirEncabezado()**, **imprimirPrecio()**, **imprimirNombreProducto()**.

### Ejercicio 8:

#### Bad smells:

- **Duplicated Code**

```
this.words
    .stream()
    .mapToLong(w -> w.length())
    .sum()
```

- Uso de Variables Temporales innecesarias

### Solución:

### Solución:

Aplicar **Extract Method** sobre el código en común.

Aplicar **Inline Temp**

**8.3)** El error es que nos puede ocurrir una división por 0, sigue presente luego de los refactorings debido a que cuando refactorizamos **NO** modificamos el comportamiento del código.

### Ejercicio 9:

**9.d)** Último inciso, apliqué **Replace Temp with Query** sobre añosDesdeFechaAlta, la query es `cliente.calcularAños()` y **Extract Method** en el cálculo del descuento.

//Consultar